

SZAKDOLGOZAT



MISKOLCI EGYETEM

SPAM/HAM üzenetek osztályozása mélytanulási módszerekkel

Készítette:

Ferencsik Róbert

Programtervező informatikus

Témavezető:

Dr. Baksáné Dr. Varga Erika

MISKOLC, 2026

SZAKDOLGOZAT FELADAT

Ferencsik Róbert (BQLOTW) BSc programtervező informatikus jelölt részére.

A szakdolgozat tárgyköre: természetes nyelvfeldolgozás, szövegosztályozás, mélytanulás

A szakdolgozat címe: SPAM/HAM üzenetek osztályozása mélytanulási módszerekkel

A feladat részletezése:

A hallgató feladata egy SPAM/HAM szövegosztályozó rendszer megtervezése és megvalósítása mélytanulási módszerekkel. A dolgozatnak tartalmaznia kell:

1. A szövegosztályozás elméleti hátterének bemutatását. A hagyományos és mélytanulási módszerek összevetését és a szekvenciális modellek szerepét.
2. A felhasznált adathalmaz részletes elemzését.
3. Az előfeldolgozási folyamat kidolgozását és implementálását.
4. Egy kétirányú LSTM (BiLSTM) modell megtervezését és megvalósítását.
5. Az elvégzett kísérletek és a kapott eredmények bemutatását.
6. A módszer korlátainak és továbbfejlesztési lehetőségeinek áttekintését.

Témavezető(k): Dr. Baksáné Dr. Varga Erika, egyetemi docens
Konzulens(ek):

A feladat kiadásának ideje: 2025. szeptember 9.

.....
szakfelelős

EREDETISÉGI NYILATKOZAT

Alulírott **Ferencsik Róbert**; Neptun-kód: BQL0TW a Miskolci Egyetem Gépészmérnöki és Informatikai Karának végzős Programtervező informatikus szakos hallgatója ezennel büntetőjogi és fegyelmi felelősségem tudatában nyilatkozom és aláírásommal igazolom, hogy *SPAM/HAM üzenetek osztályozása mélytanulási módszerekkel* című szakdolgozatom saját, önálló munkám; az abban hivatkozott szakirodalom felhasználása a forráskezelés szabályai szerint történt.

Tudomásul veszem, hogy szakdolgozat esetén plágiumnak számít:

- szószerinti idézet közlése idézőjel és hivatkozás megjelölése nélkül;
- tartalmi idézet hivatkozás megjelölése nélkül;
- más publikált gondolatainak saját gondolatként való feltüntetése.

Alulírott kijelentem, hogy a plágium fogalmát megismertem, és tudomásul veszem, hogy plágium esetén szakdolgozatom visszautasításra kerül.

Miskolc, év hó nap

.....
Hallgató

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- | | |
|------------------------------|-----------------------------|
| témavezető (dátum, aláírás): | konzulens (dátum, aláírás): |
| | |
| | |
| | |

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- | | |
|-------|---------------|
| | |
| dátum | témavezető(k) |

- dátum szakfelelős

- Miskolc,
a Záróvizsga Bizottság Elnöke

Tartalomjegyzék

1. Bevezetés	1
2. Szövegosztályozási feladat és módszerek	3
2.1. Természetes nyelvfeldolgozás	3
2.2. Szövegosztályozás	3
2.3. A szövegosztályozás folyamata	4
2.4. Áttérés a hagyományos módszerekről a mélytanulási módszerekre . . .	4
2.5. Felhasznált módszerek	6
2.5.1. Előfeldolgozás	6
2.5.2. Modell	8
3. Előfeldolgozás	11
3.1. Bevezetés a fejezet felépítéséhez	11
3.2. Adathalmaz	11
3.2.1. Adathalmaz eredete	11
3.2.2. Adathalmaz szerkezete és értékei	12
3.3. Előfeldolgozás tervezése	13
3.3.1. Előfeldolgozás szükségessége	13
3.3.2. Előfeldolgozási folyamat	14
3.3.3. Előfeldolgozás implementálása	14
3.3.4. Feltáró adatelemzés, adattisztítás és ellenőrzés	15
3.3.5. Tokenizálás és validálása	17
4. A klasszifikációs modell és tanítási folyamat	20
4.1. Modelltervezés	20
4.1.1. Modell architektúra	20
4.1.2. Hiperparaméter-tér és optimalizálási stratégia	21
4.2. Implementáció	23
4.2.1. Állomány és konfigurációk kezelése	24
4.2.2. Modell, hiperparaméterek és tanulási görbe	25
5. Kísérletek és eredmények	26
5.1. Előfeldolgozás és tokenizálás	26
5.2. Hiperparaméter-optimalizálás és tanulási görbe elemzés	26
5.2.1. Kísérleti környezet, metrika és döntési szabály	27
5.2.2. A három fázis keresési tere	27
5.2.3. Első fázis: széles keresési tér és megszakított futás	28
5.2.4. Második fázis: szűkített keresési tér és stabilabb tuning	28
5.2.5. Harmadik fázis: konzervatívabb tér és tanulási görbe	29

5.2.6.	A 0.5-ös küszöb és az instabil konfigurációk hatása	30
5.2.7.	Összegzés: kompromisszumok és tanulságok	31
5.2.8.	Korlátok és továbbfejlesztési lehetőségek	31
6.	Összefoglalás és továbbhaladási irány	32
	Források	33

1. fejezet

Bevezetés

Az információs technológiák rohamos fejlődése alapjaiban alakította át a kiberbiztonságot. Az új technológiák megjelenésével ugyan jelentősen bővült a védekezés eszköztára, ez azonban korántsem jelenti a fenyegetések megszűnését vagy csökkenését. A digitális térben zajló „küzdelem” inkább egy folyamatosan változó, dinamikus folyamatként írható le, ahol a támadási és védekezési módszerek egymással párhuzamosan fejlődnek. Az internet széles körű elterjedése egy korábbi jelentős fordulópontot jelentett eme küzdelem történetében, áthelyezve színterét, míg napjainkban a mesterséges intelligencia új eszközöket biztosít mind a támadók, mind a védekezők számára.

A köztudatban élő kép, miszerint a kiberbiztonság valós idejű „programozói párbajként” értelmezhető, nem tükrözi a valóság összetettségét. A gyakorlatban a hangsúly sokkal inkább a már működő rendszerekben található sérülékenységek feltárásán és kezelésén van, hogy a támadók ne is férjenek hozzá olyan felülethez, ahol kódot tudnak futtatni. Ez a védelem nem csupán a forráskód minőségétől függ, hanem számos egyéb tényező együttes hatásának eredménye. Ilyen tényező például a titkosítás, a megfelelő jogosultságkezelés, a helyes konfigurációk, és az emberi figyelem és tudás.

Az egyik leggyakoribb támadási forma az **adathalászat** (phishing). Az ilyen típusú támadások során a támadók megtévesztő, gyakran hitelesnek tűnő üzenetek segítségével próbálnak érzékeny adatokat megszerezni a felhasználóktól. Az adathalász e-mailek különösen veszélyesek, mivel kihasználják a felhasználók figyelmetlenségét, illetve döntéshozatali korlátaikat. Mindez rámutat arra, hogy a kizárólag felhasználói döntésekre építő védekezés nem tekinthető elegendőnek.

Ebből következően a modern kiberbiztonsági megoldások egyik kulcseleme az automatizált védelem alkalmazása, mely a valóságban a természetes nyelv terén, illetve tapasztalat alapján egyéb területeken is inkább nevezhető félautomatának. Az e-mail szolgáltatók fejlett szűrőrendszereket használnak annak érdekében, hogy a nem kívánt vagy kártékony üzeneteket még a felhasználóhoz való eljutás előtt azonosítsák és elküldönítsék habár ez nem jelenti, hogy nem kerül kéretlen üzenet a levélfiókba.

Ezt a folyamatot klasszifikációnak, osztályba sorolásnak tekintjük. Gépi módszerek számos fajtája alkalmazható, és ugyancsak sokféle adatból lehet döntést hozni arról, hogy egy e-mail kéretlen-e. A szöveg alapú megközelítések mellett a metaadatokból, hálózati jellemzőkből és viselkedési mintákból automata vagy félautomata módon információt kinyerő modellek és ezek konkrét megvalósítása végtelen számú megoldást nyújt a feladatra.

A beérkező üzenetek nagy mennyisége és folyamatosan növekvő változatossága miatt a modern rendszerek jellemzően **gépi tanulási** (Machine Learning) módszerekre

épülnek, amelyek képesek alkalmazkodni az újonnan megjelenő mintázatokhoz, valamint a dinamikusan változó és egyre kifinomultabb támadási technikákhoz. E megközelítések közül különösen kiemelkednek a **mélytanulási** (Deep Learning, DL) módszerek, amelyek a hagyományos gépi tanulási eljárásokhoz képest számos jelentős előnnyel rendelkeznek.

A mélytanulási modellek egyik legfontosabb tulajdonsága, hogy jelentősen csökkentik a kézi **jellemzőtervezés** (Feature Engineering) szükségességét, amely a klasszikus módszerek esetében gyakran időigényes, szakértelmet igénylő folyamat. Ezzel szemben a mély neurális hálózatok képesek közvetlenül a nyers adatokból automatikusan kinyerni a releváns jellemzőket és mintázatokat, ezáltal a tanulási folyamat sokkal inkább adatvezéreltté és kevésbé emberi beavatkozást igénylővé válik. Ennek eredményeként a modell nemcsak hatékonyabban képes kezelni a nagy mennyiségű és komplex adatokat, hanem jobb általánosító képességet is mutathat új, korábban nem látott példák esetén. Egy nagy hátrányt érdemes megemlíteni, a mélytanulási módszerek számítási költsége jelentős. Az egyik megközelítés a **kéretlen levél** (spam) és **valódi levél** (ham) kategóriák osztályba sorolására a természetes nyelvű szekvenciák vizsgálata. Ezen dolgozat ezt veszi alapul, és vizsgálja mélytanulási módszerekkel. Időben vagy térben folytonos sorrendű adatok feldolgozására már korlátozottabb az eszköztár. A természetes nyelvi szekvenciák feldolgozására különösen alkalmasak a **hosszú-rövid távú memória** (Long Short-Term Memory, LSTM) architektúrán alapuló visszacsatolt neurális hálózatok, amelyek a nyelvben előforduló statisztikai mintázatokat tanulják meg. A szöveges e-maileket adat sorozatként kezelik, ahol az egyes szavak és nyelvi mintázatok jelentése nagyban függ az őket megelőző kontextustól. A neurális hálózat a korábban látott szókapcsolatok és statisztikai összefüggések alapján tanulja meg felismerni azokat a mintákat, amelyek jellemzőek lehetnek a kéretlen vagy a valódi levelekre. Az osztályozási döntés során nem kizárólag egyes kulcsszavak jelenlétét vizsgálja, hanem azok sorrendjét, gyakoriságát és környezetét is figyelembe veszi. Ennek köszönhetően a modell képes összetettebb nyelvi és stilisztikai mintázatok felismerésére is, amelyek hozzájárulnak az e-mailek pontosabb osztályozásához.

A dolgozat fókusza a hosszú-rövid távú memória (LSTM) architektúrán alapuló szövegosztályozás gyakorlati vizsgálata. Az LSTM-alapú megközelítés a dolgozat során implementációval és kísérletekkel kerül bemutatásra, beleértve a modell felépítését, tanítását és kiértékelését.

A kutatás további célja a modell megfelelő felépítésének és optimális paraméterezésének kialakítása mellett az előfeldolgozási lépések részletes és szisztematikus meghatározása is, mivel ezek alapvetően befolyásolják a tanítás hatékonyságát és a végső teljesítményt. Kiemelt szempont a modell pontosságának és egyéb teljesítménymutatóinak átfogó vizualizálása és elemzése, amely lehetővé teszi a különböző konfigurációk közötti összehasonlítást és a tanulási folyamat viselkedésének jobb megértését. Emellett a dolgozat egyik fontos vizsgálati iránya annak feltárása is, hogy mekkora mennyiségű tanítóadat szükséges egy stabil, megbízhatóan általánosító modell eléréséhez, különös tekintettel arra, hogyan változik a teljesítmény a növekvő adatmennyiség függvényében.

2. fejezet

Szövegosztályozási feladat és módszerek

Ebben a fejezetben a kontextus megteremtése érdekében a természetes nyelvfeldolgozás és a szövegosztályozás alapfogalmai kerülnek bevezetésre. Ezt követően a szövegosztályozás folyamatáról esik szó. Végül a felhasznált módszerek részletezése következik.

2.1. Természetes nyelvfeldolgozás

A természetes nyelvfeldolgozás olyan tudományág, amely lehetővé teszi a számítógépek és digitális rendszerek számára az emberi nyelv értelmezését, megértését és generálását. Integrálja a számítógépes nyelvészetet, a szabályalapú megközelítéseket, a statisztikai modellezést, valamint a korszerű mélytanulási technikákat [13].

A természetes nyelvfeldolgozás területén folyó kutatások döntő szerepet játszottak a modern nyelvtechnológiai rendszerek fejlődésében, elősegítve a szöveges és beszélt nyelv értelmezésére, generálására és összetett feladatok végrehajtására képes megoldások kialakulását [13].

2.2. Szövegosztályozás

A szövegosztályozás egy olyan folyamat, amely során egy dokumentumot automatikusan egy vagy több kategóriába sorolnak. Létezik felügyelt és nem felügyelt változata.

A **felügyelt gépi tanulás** (Supervised Learning) módszer alapja a dokumentum belső jellemzőinek elemzése, majd egy előre címkézett adathalmaz segítségével modell létrehozása a dokumentum és a kategória közötti kapcsolatok feltérképezésére [16].

A szövegosztályozás jelentősége az **adatrobbanás korszakával** (Era of Big Data) tovább nőtt, hiszen lehetővé teszi a strukturálatlan szöveges adatok hatékony szervezését és feldolgozását. Számos tanulmány rámutat arra, hogy a szövegosztályozás nem csupán egy a sok természetes nyelvfeldolgozás feladat közül, hanem a terület alapvető építőeleme. [8] megállapítja, hogy „a szövegbesorolás a természetes nyelvfeldolgozás legfontosabb és legalapvetőbb feladata” (saját fordítás).

2.3. A szövegosztályozás folyamata

A folyamat tipikusan több egymást követő lépésből áll, melyet a 2.1. ábra szemléltet. Kezdve a **szöveg előfeldolgozásával** (preprocessing), amely magában foglalja a tokenizálást, a stop-szavak eltávolítását, a szöveg kisbetűsítését, valamint a szótövezést vagy a lemmatizálást. Az előfeldolgozás célja egy egységes, zajtól megtisztított és könnyen feldolgozható szövegállomány előállítása, amely stabil alapot biztosít a további lépésekhez. Ezt követi a **jellemzőkinyerés** (feature extraction), amelynek során a dokumentumok numerikus reprezentációra alakíthatók. Ez a reprezentáció teszi lehetővé, hogy a különböző gépi tanulási algoritmusok a szöveg struktúráját és tartalmi jellemzőit kvantitatív módon értelmezzék.



2.1. ábra: A szövegosztályozás tipikus folyamata gépi tanulási módszerekkel. *Forrás:* [8].

Az osztályozási modell létrehozása előtt általában sor kerül a tanított modell **kiértékelésére** (evaluation), amely során különböző teljesítménymutatók – például pontosság, visszahívás, F1-mérték vagy **görbe alatti terület** (Area Under the Curve, AUC) – segítségével vizsgálható, hogy a modell milyen mértékben képes a különböző kategóriák helyes elkülönítésére. A kiértékelés elengedhetetlen lépés annak meghatározásához, hogy a modell alkalmas-e éles környezetben történő alkalmazásra, illetve hogy szükség van-e további finomhangolásra vagy alternatív jellemzők használatára.

A folyamat végén az **osztályozó** (classifier) a dokumentumot a megfelelő **címkéhez** (label) rendeli, amely a későbbi elemzési feladatok kiindulópontja lehet. A szövegosztályozás eredményei többféle további feldolgozást támogatnak, például **érzelmi állapotok feltárását** (sentiment analysis), **témaazonosítást** (topic analysis), spam- vagy anomáliaészlelést, valamint nagy szövegtörzsek strukturálását és információ-visszakeresési rendszerek hatékonyságának javítását. A címkézett adatok ezen túlmenően alapot biztosítanak prediktív modellekhez, ajánlórendszerekhez, illetve bármely olyan automatizált folyamat számára, ahol nagy mennyiségű szöveges adat gyors és megbízható kategorizálása szükséges.

2.4. Áttérés a hagyományos módszerekről a mélytanulási módszerekre

[8] szerint „az 1960-as évektől a 2010-es évekig a hagyományos szövegbesorolási módszerek domináltak” (saját fordítás), és a mélytanulási módszerekre való áttérés 2010-ben kezdődött. A hagyományos módszerek statisztikán alapulnak, és a korábbi szabályalapú módszerekhez képest nagyobb pontosságot és stabilitást mutattak. Az adatrobbanás

korában azonban a manuális jellemzőkinyerés költséges és időigényes, a statisztikai megközelítések pedig figyelmen kívül hagyják a szöveges adatok természetes szekvenciális szerkezetét vagy kontextusbeli információit.

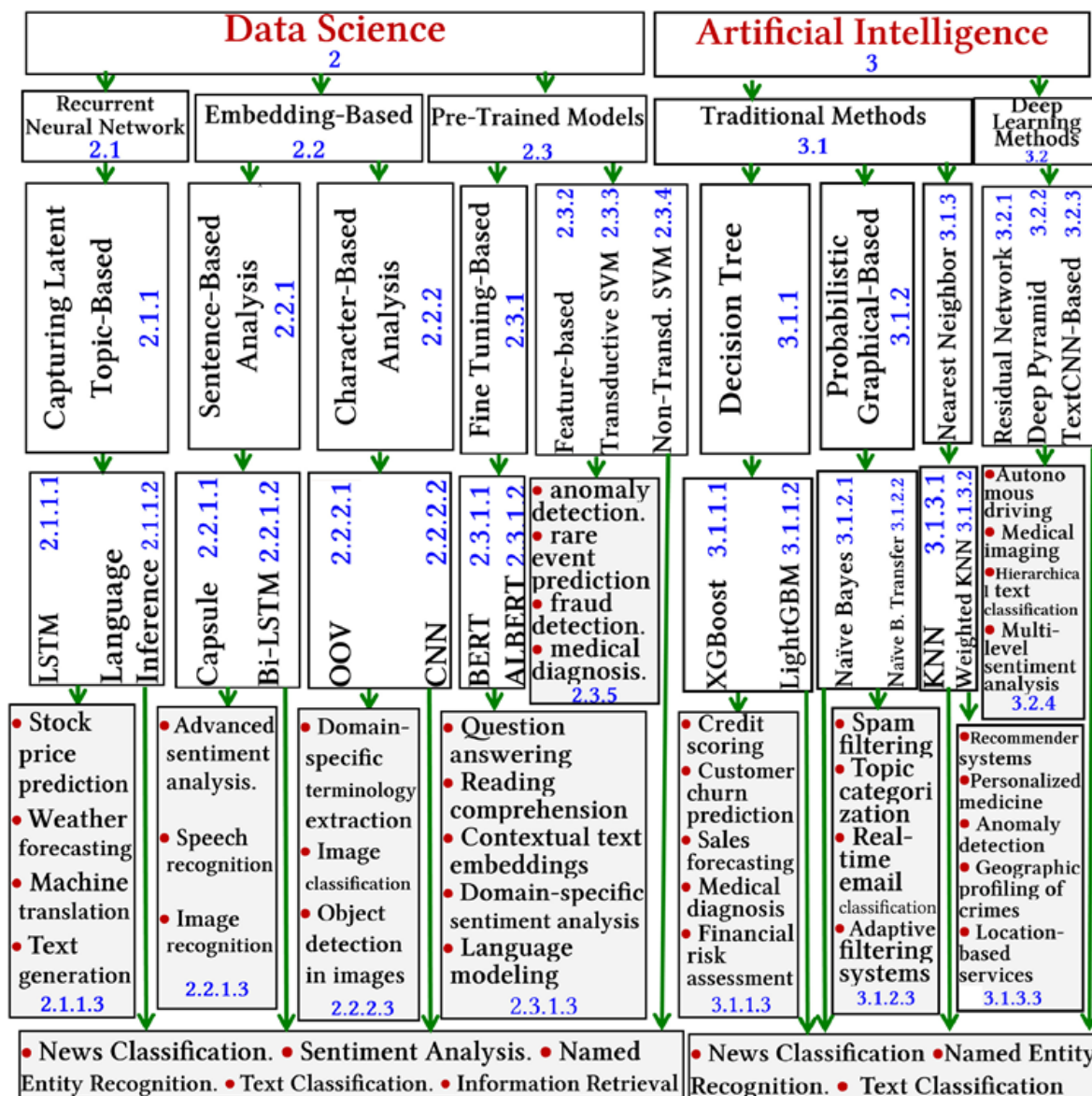
A hagyományos megközelítésekkel szemben a mélytanulási módszerek kiküszöbölik a kézzel készített szabályok és kézi jellemzőkinyerés szükségességét, mivel képesek az adatokból automatikusan, szemantikailag gazdag és kontextusérzékeny reprezentációkat tanulni. Ez a tulajdonság lehetővé teszi a **mély neurális hálózatok** (Deep Neural Network, DNN) számára, hogy olyan összetett nyelvi mintázatokat, hosszú távú függőségeket és finom szemantikai különbségeket is megragadjanak, amelyeket a hagyományos technikák jellemzően nem képesek megfelelő pontossággal modellezni. Ennek következtében a szövegosztályozás területén végzett jelenkori kutatások döntő része mély neurális hálózatokra épül, amelyek adatvezérelt architektúrákként automatikusan képesek hierarchikus jellemzők kinyerésére, és számos alkalmazási területen a legkorszerűbb teljesítményt biztosítják. Ugyanakkor a mélytanulási modellek hatékony működéséhez általában jelentős számítási kapacitásra és nagy méretű, annotált tanítóadatokra van szükség. Emiatt a **hagyományos gépi tanulási** (Traditional Machine Learning) módszerek iránti érdeklődés elsősorban azokban a scenáriókban maradt fenn, ahol a számítási erőforrások korlátozottak, az adatmennyiség csekély, vagy valós idejű feldolgozási követelmények teszik szükségessé a könnyebb, kompaktabb modellek alkalmazását.

A szövegosztályozási technikák két nagy csoportba sorolhatók: **adattudomány** (Data Science) és **mesterséges intelligencia** (Artificial Intelligence) [14].

Itt érdemes megemlíteni, hogy ez a két osztály nem diszjunkt, hanem az adattudomány egy szélesebb terület, amelynek egyik eszköze lehet a mesterséges intelligencia. Ezt a következtetést definíciójukból le lehet vonni.

Az adattudomány: „Olyan tudományág, amely tudományos módszereket, folyamatokat és rendszereket alkalmaz az adatokból származó ismeretek és betekintések kinyerésére” [15] (saját fordítás).

A mesterséges intelligencia egyik definíciója: „Egy technológia, amely felruházza a számítógépeket és gépeket az emberi tanulás, megértés, problémamegoldás, döntéshozatal, kreativitás és önállóság utánzásának képességével” [3] (saját fordítás).

2.2. ábra: A szövegosztályozási módszerek taxonómiája. *Forrás:* [14].

2.5. Felhasznált módszerek

A felhasznált módszerek két csoportban kerülnek bemutatásra. Az első csoport az adathalmaz feldolgozására vonatkozik, míg a második a hosszú-rövid távú memória neurális háló paramétereire és konkrét típusára. Az egyszerűbb és közismertebb módszerek nem kerülnek bemutatásra, az implementáció részletezésénél felsorolásra kerülnek.

2.5.1. Előfeldolgozás

Tukey interkvartilis tartomány

Az adathalmazt tekintve az üzenetek hossza egy nagyon eltérő tartományon oszlik el. A rövid, pár szavas üzenetektől egészen a hosszú üzenetváltásokig, melyek akár százazres nagyságrendbe is lépnek karakterszám tekintetében. A döntés meghozatala,

hogymelyik üzenet túl rövid, és melyik túl hosszú, statisztikai alapon fekszik. Méghozzá **Tukey interkvartilis módszerén** (Tukey's Interquartile Range Method, Tukey's IQR-method).

Az **interkvartilis tartomány** (interquartile range, IQR) egy statisztikai mutató mely információt ad az adatok eloszlásáról, azonban egyike azon módszereknek melyet nem befolyásol jelentősen az adatok nem normális jellege, esetünkben az eloszlás **ferdesége** (skewness). A harmadik (Q_3) és első kvartilis (Q_1) különbsége adja értékét [11].

$$IQR = Q_3 - Q_1$$

A Tukey-módszer alapján a kiugró értékek az alábbi határok segítségével határozhatók meg:

$$\text{alsó határ} = Q_1 - 1.5 \cdot IQR$$

$$\text{felső határ} = Q_3 + 1.5 \cdot IQR$$

Entitások helyettesítése

Az **entitások helyettesítése** (Entity Masking) számos természetesnyelv-feldolgozási feladat egyik fontos előfeldolgozási lépése. Az eljárás során a **megnevezett entitások** (Named Entities), például személynevek, telefonszámok, e-mail címek, dátumok vagy helynevek helyére előre definiált helyettesítő tokenek kerülnek, miközben az adott elem szemantikai szerepe továbbra is megmarad a szövegben. A módszer célja feladattól függően eltérő lehet, a jelen dolgozat esetében azonban elsősorban a szókészlet méretének csökkentése volt a cél. Az entitások helyettesítésével a modell kevésbé a konkrét karakterláncokat, sokkal inkább azok kontextuális szerepét és a szövegben megjelenő mintázatokat tanulja meg.

Az entitásmaszkolás további természetesnyelv-feldolgozási feladatok során is széles körben alkalmazott technika. Gyakori felhasználási területe az adatvédelem, ahol szükséges a személyes vagy érzékeny adatok eltávolítása a modellek tanítása előtt. Emellett jelentős szerepet játszik a szókészlet csökkentésében és a szöveges adatok normalizálásában is, mivel az azonos típusú entitások egységes reprezentációt kapnak. Ez javíthatja a modellek generalizációs képességét, mivel a hálózatok kevésbé a konkrét értékeket memorizálják, hanem az azokhoz kapcsolódó nyelvi mintázatokat és kontextusfüggéseket sajátítják el. A generatív és előtanított nyelvi modellek esetében a maszkolás az előtanítás egyik fontos eleme is lehet, ahol a modellek feladata a maszkolt tokenek predikciója. Az ilyen jellegű tanítás lehetővé teszi, hogy a modellek mélyebb szemantikai és szintaktikai reprezentációkat tanuljanak a nyelvi adatokból.

SentencePiece tokenizálás

A Kudo és Richardson által 2018-ban bemutatott SentencePiece egy nyelvfüggetlen résszavas tokenizáló rendszer mely képes nyers, zajos szövegekből szókészlet-tanulásra.

A működésének alapja egy statisztikai szegmentáló algoritmus alkalmazása a szöveg apróbb egységekre bontására, előfeldolgozás (tokenizálás, kisbetűsítés stb.) szükségése nélkül. A teljes betanítás során a beviteli szövegben található karakterláncokat közvetlenül a modell megtanulja, nem kell előre tokenizált szavakkal dolgozni. Ennek

eredményeként a SentencePiece nyelvfüggetlen feldolgozást tesz lehetővé: ugyanazon algoritmus alkalmazható bármilyen nyelven, akár szóközzel tagolt (angol), akár összefüggő (japán, kínai) szövegeken is, és egyetlen, előre rögzített méretű szótárra építve hozza létre a tokeneket [7].

A módszer további előnye a veszteségmentes tokenizáció, mellyel a különleges karakterek, sőt még a szóközök információja is megmarad. Így visszaállítható az eredeti szöveg és bármikor biztonságosan reprodukálható a tokenizációs folyamat. Illetve alkalmazása jelentősen csökkenti a hagyományos jellemzőtervezés szükségességét, és elősegíti a modellek számára a releváns mintázatok tanulását [7].

E-mail (spam/ham) osztályozáskor a SentencePiece különösen előnyös a szövegek zajossága és változatossága miatt. A spam üzenetek gyakran tartalmaznak elgépeléseket, felesleges karaktereket, melyeket szótári módszerekkel nehéz és költséges kezelni. Mindez robusztusabbá teszi a rendszer működését, és a félreírt, felcserélt vagy egymáshoz fűzött szóalakokat sem kezeli „ismeretlenként”. A résszavas bemenet ráadásul a modern neurális hálózatok számára is ideális.

Az irodalomban nem sűrűn fordul elő, ha egyáltalán előfordul, a SentencePiece használata hosszú-rövid távú memória modellek esetében. Ennek oka a két módszer megjelenésének időpontja és aktív használata lehet. A hosszú-rövid távú memória modellek korszakában, körülbelül 2015-től 2019-ig még a Word2Vec, GloVe és a sima szó-tokenizálás volt a jellemző, míg a SentencePiece elsősorban a Transformer, BERT, többnyelvű módszerekkel lett népszerű.

2.5.2. Modell

Kétirányú hosszú-rövid távú memória neurális háló

A **kétirányú hosszú-rövid távú memória** (Bidirectional Long Short-Term Memory, BiLSTM) neurális hálózat a **rekurrens neurális hálók** (RNN) egy továbbfejlesztett változata. A klasszikus LSTM modellekkel ellentétben, amelyek a szöveget csak egy irányban (balról jobbra) dolgozzák fel, a BiLSTM architektúra két különálló LSTM réteget alkalmaz. Az egyik a bemeneti szekvenciát előre, a másik pedig visszafelé dolgozza fel. Ennek eredményeként a modell minden token esetében hozzáfér mind az előző, mind a következő kontextushoz, ami jelentősen javítja a szöveges reprezentáció minőségét [19].

Szövegklasszifikációs feladatok esetében ez különösen előnyös, mivel a szavak jelentése erősen függ a környező tokenektől, és gyakran a későbbi kontextus is szükséges a helyes értelmezéshez. 2016-ban kimutatták, hogy a kétirányú hosszú-rövid távú memória modellek hatékonyabban képesek modellezni a hosszabb távú összefüggéseket a szövegben mint az egyirányú hosszú-rövid távú memória modellek [9].

További módszerekkel még pontosabb eredményeket lehet elérni, például a **figyelem** (attention) mechanizmusának alkalmazásával, azonban ezekre a módszerekre a jelen dolgozat nem tér ki.

Véletlenszerű hiperparaméter-keresés

A **véletlenszerű hiperparaméter-keresés** (Hyperparameter Optimization with Random Search) egy népszerű hiperparaméter-optimalizálási módszer, amely a **rácskereséssel** (Grid Search) ellentétben nem egy előre meghatározott rácsos elosztást vizsgál, hanem az előre megállapított tartományból véletlenszerűen mintavételez.

Bergstra és Bengio szerint a véletlenszerű keresés hatékonyabb, mint a rácskeresés mivel hasonló teljesítményű beállítást talál a modellnek az idő töredéke alatt, és jelentősen kevesebb számítási kapacitást igényel. Ugyanakkora számítási erőforrás mellett a véletlenszerű próbálkozás több különböző kombinációt fed le, így nagyobb eséllyel talál magára a fontos beállításokra. Szóval a véletlenszerű keresés megtartva a rácskeresés előnyeit, (egyszerű implementálás, egyértelmű párhuzamosítás), magas dimenziójú feladatokban komoly hatékonyság növekedést eredményez [1].

Az adott feladathoz egyszerűsége és gyors implementálhatósága elengedhetetlen volt, mivel adott kérdésekre ad választ a dolgozat, nem egy gyártásba tervezett módszer, így az időkölség volt az első szempont. Ez a módszer pedig megfelelő értékeket ad a hiperparaméterek optimalizálására.

Kiértékelés

A modell kiértékelésére több klasszikus mérőszám áll rendelkezésre, amelyek alkalmasak a szövegklasszifikációs feladatok kiértékelésére. A vizsgálat részeként egy **kumulatív tanítási görbe** (Learning Curve) is alkalmazásra került, mely a tanító adat fokozatos bővítésével mutatja a modell teljesítményének alakulását az F1-pontszám alapján.

Ez a megközelítés lehetővé teszi annak vizsgálatát, hogy a rendelkezésre álló adatmennyiség növekedése hogyan befolyásolja a modell általánosító képességét. A görbe így nem a tanítási folyamat konvergenciáját, hanem az adatmennyiség és a modell pontossága közötti kapcsolatot szemlélteti. Amennyiben a görbe telítődést mutat, az arra utalhat, hogy a további adatmennyiség már nem eredményez jelentős javulást a modell F1-pontszám értékében.

A klasszifikációs teljesítmény részletesebb elemzésére továbbá rendelkezésre áll az úgynevezett **konfúziós mátrix** (Confusion Matrix), mely a valós és prediktált osztályok közötti összefüggéseket mutatja. A mátrix elemei a következők:

	Prediktált pozitív	Prediktált negatív
Valós pozitív	Igaz Positív	Hamis Negatív
Valós negatív	Hamis Positív	Igaz Negatív

- **Igaz Positív** (True Positive, TP): a modell helyesen spamnek (pozitív osztály) jelöl egy valóban spam üzenetet.
- **Igaz Negatív** (True Negative, TN): a modell helyesen hamnek (negatív osztály) jelöl egy valóban nem spam (ham) üzenetet.
- **Hamis Positív** (False Positive, FP): a modell spamnek jelöl egy valójában ham üzenetet (téves riasztás).
- **Hamis Negatív** (False Negative, FN): a modell hamnek jelöl egy valójában spam üzenetet (el nem kapott spam).

A konfúziós mátrix alapján került kiszámításra a pontosság (precision), a visszahívás (recall) és az F1-érték (F1-score) [4].

A pontosság azt mutatja meg, hogy a pozitívnak prediktált minták közül mennyi volt valóban pozitív, amelyet az 2.1 egyenlet definiál.

$$\text{Pontosság} = \frac{TP}{TP + FP} \quad (2.1)$$

A visszahívás azt fejezi ki, hogy a tényleges pozitív minták hány százalékát sikerült helyesen azonosítani, amelyet az 2.2 egyenlet ír le.

$$\text{Visszahívás} = \frac{TP}{TP + FN} \quad (2.2)$$

Az F1-érték a pontosság és visszahívás harmonikus átlaga, amely kiegyensúlyozott teljesítménymutatót biztosít, különösen osztály-egyensúlytalanság esetén [4]. Az F1-érték számítását az 2.3 egyenlet mutatja be.

$$F1 = 2 \cdot \frac{\text{Pontosság} \cdot \text{Visszahívás}}{\text{Pontosság} + \text{Visszahívás}} \quad (2.3)$$

3. fejezet

Előfeldolgozás

3.1. Bevezetés a fejezet felépítéséhez

Az előfeldolgozásról szóló fejezetben bemutatásra kerül az adathalmaz rövid történeti háttere, valamint az előfeldolgozási folyamatok. A fejezet ismerteti a végrehajtás lépéseit a nyers, vesszővel tagolt CSV formátumú üzenetekről kezdve egészen a tanítási, validációs és tesztelési adathalmazok előállításáig, beleértve a tokenizáló modell elkészítését is.

Az implementációt ismertető részekben a hangsúly a „miért” és a „hogyan” kérdéseken van, nem pedig a teljes kódbázis részletes bemutatásán. A futtatható kód a verziókezelte forráskódban és a Jupyter notebookokban érhető el, míg a dolgozat első sorban a döntések indoklását és a be- és kimenetek közötti kapcsolat dokumentálását tartalmazza.

3.2. Adathalmaz

3.2.1. Adathalmaz eredete

A felhasznált adathalmaz M. Wiechmann `enron_spam_data` GitHub-oldalán található vesszővel tagolt adatfájl (CSV) formátumban [17].

Az eredeti adathalmazról, mely abban különbözik a leírtak szerint, hogy minden egyes e-mail külön fájlként van tárolva, a *Spam filtering with Naive Bayes – Which Naive Bayes?* tanulmány harmadik fejezetében olvashatunk. A következő bekezdések ezt a művet veszik alapul az adathalmaz általános bemutatására [10].

Az Enron botrány kivizsgálásának keretében közel 150 Enron dolgozó fájljai lettek közzé téve, melyek között jelentős mennyiségű személyes e-mail üzenet is megtalálható. Személyre szabott spam szűrőfejlesztéséhez kiválasztottak 6 Enron dolgozót: `farmer-d`, `kaminski-v`, `kirchen-l`, `williams-w3`, `beck-s` és `lokay-m`, a Bekkerman által szolgáltatott adathalmazból, melyben a levelezések csak ham üzeneteket tartalmaztak [10].

A spam üzeneteket négy különböző forrásból állították össze, melyek sorra: a SpamAssassin corpus, a Honeypot project, Bruce Guenter spam gyűjteménye, és Paliouras G. egyéni spam gyűjteménye. A hat darab ham üzenet gyűjteményből mindegyik párba lett állítva egy spam gyűjteménnyel a három közül, és az arányuk három esetben 1:3-hoz lett, míg a másik három esetben 3:1-hez [10].

A közzétett korpusz készítői az alábbi előfeldolgozási lépéseket is elvégezték:

- az e-mail fiók tulajdonosa által küldött levelek eltávolítása;
- HTML címkék eltávolítása;
- e-mail fejléc eltávolítása;
- nem latin karaktereket alkalmazó spamek eltávolítása.

3.2.2. Adathalmaz szerkezete és értékei

Összesen 33.716 üzenet található az adathalmazban kéretlen és valódi válogatás nélkül. Az adathalmaz öt oszlopban tárolja az adatokat egyes üzenetváltásokról. Ez az öt oszlop sorra az index, a tárgy, az üzenet törzse, a címke és a dátum. Eredeti néven sorra: Message ID, Subject, Message, Spam/Ham és Date. Az első bejegyzés a fejléc információkkal együtt a következő:

```
Message ID,Subject,Message,Spam/Ham,Date
0,christmas tree farm pictures,,ham,1999-12-10
```

A címkék egyensúlyban vannak, csupán egy százalékponttal tér el a két osztály aránya. Számszerűsítve a spam üzenetek száma 17.171 azaz 50.93%, míg a ham üzenetek száma 16.545 azaz 49.07%. Ez fontos információ a modellezés során, mivel így a **torzítás** (bias) könnyebben elkerülhető, és a pontosság kiértékelésekor nem félrevezető.

A tárgy és az üzenet törzse fontos információ, mivel ezek alapján kerül implementálásra az osztályba sorolás, ezek nyers karakteres formátumúak, és tartalmazznak metadatokat is, idő, feladó stb. Zajosak, speciális karaktereket, táblázatokat, linkeket, és az Enron céghez kapcsolódó specifikus nyelvezetet tartalmazznak. Azonban nem tartalmazznak nagybetűt, és a speciális karakterek köré szóközöket illesztettek, melyek elméletben az adatok gépi módszerrel való feldolgozását nehezítik. A címkék formátuma szöveges spam és ham alakban jelenik meg. A dátum oszlopban a bejegyzések alakja éééé-hh-nn formátumot követi.

A hiányzó adatokról, különböző mintázatokról és duplikációk számáról a következő táblázatok adnak információt.

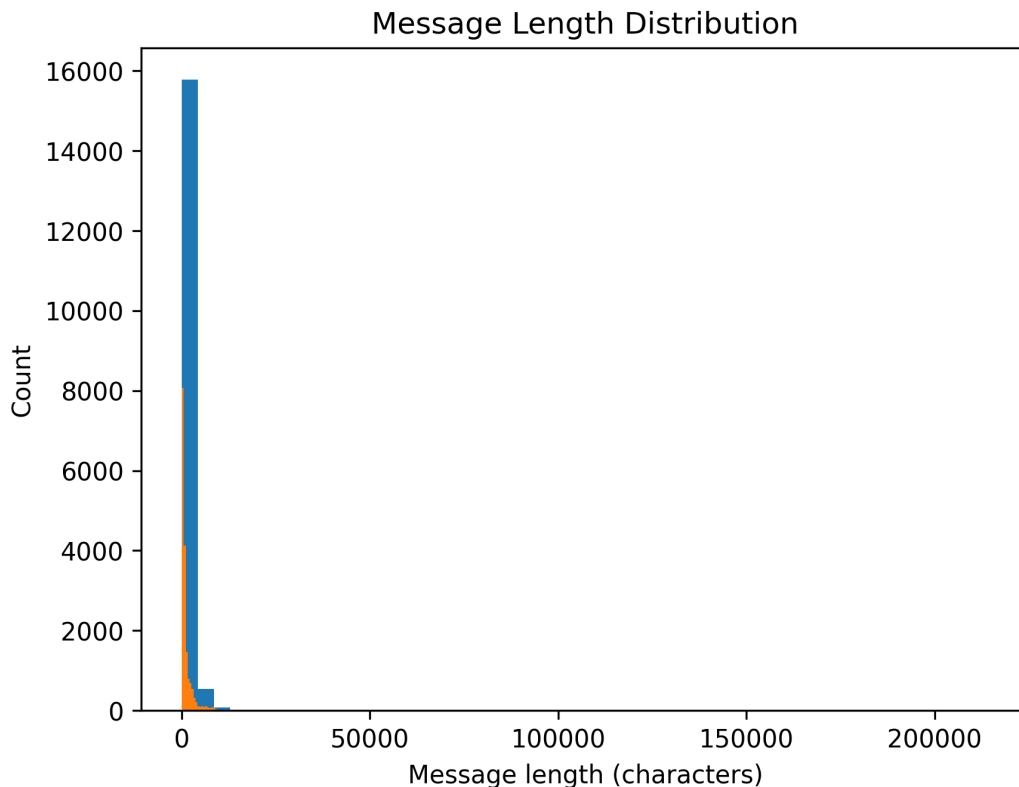
3.1. táblázat: Adathalmaz tisztasági és hiányzó érték statisztikái

Jellemző	Darabszám
Duplikált sorok száma	2953
Hiányzó Message mezők	371
Hiányzó Subject mezők	289
Hiányzó Spam/Ham mezők	0
Hiányzó Date mezők	0

3.2. táblázat: A szöveges adatokban detektált mintázatok és előfordulások

Szövegjellemző	Előfordulás
URL-ek száma	266754
E-mail címek száma	16294
Telefonszámok száma	11992
Számok összesen	435439
Ismétlődő karakterek	79316

Az üzenetek hosszának eloszlása karakter szerint erősen jobbra ferde (A 3.1. ábra), az átlag jóval nagyobb, mint a medián (átlag: 1365, medián: 613), mely azt jelenti, hogy a rövid üzenetek száma magas, azonban van néhány extrém hosszú üzenet, ami felfelé húzza az átlagot. A maximális karakterszámú üzenet 214.422 terjedelmű.



3.1. ábra: Nyers korpusz üzenethossz-eloszlása karakterszámban.

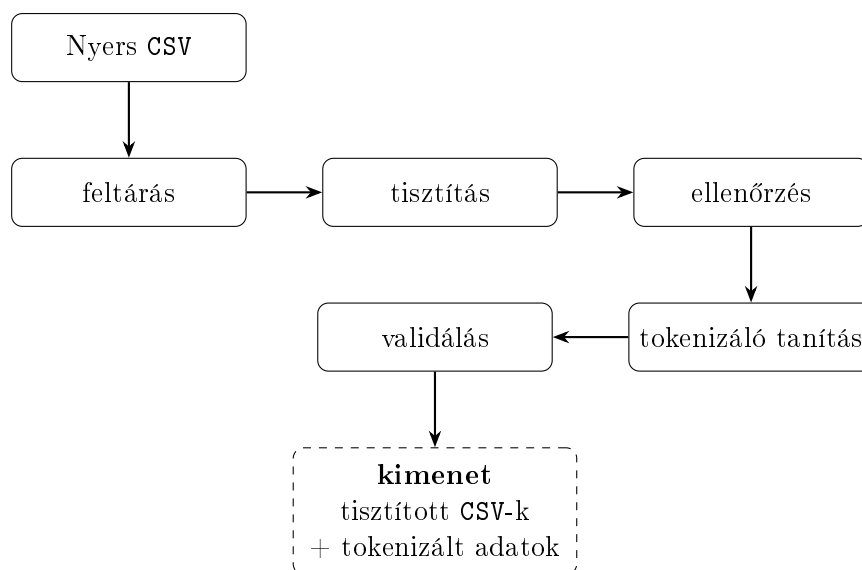
3.3. Előfeldolgozás tervezése

3.3.1. Előfeldolgozás szükségessége

Különböző feladatok és eszközök használatakor különböző kihívások és célok merülnek fel. A hosszú-rövid távú memória modellek esetében két fő célja van az előfeldolgozási lépésnek. Az egyik a zaj csökkentése, hogy a tanítás során a helyes összefüggések kerüljenek feltárára. A másik a szókincs csökkentése mely nagy összetett feladatok esetében is 30.000 maximum 50.000 token.

3.3.2. Előfeldolgozási folyamat

A 3.2. folyamatábra magas szinten mutatja be a lépések közötti kapcsolatot és alapot ad az ábrát követő magyarázatokhoz.



3.2. ábra: A lépések közötti adatáramlás. Az ábra a fájlok mozgását és a felelőségek felosztását hangsúlyozza

Az adatok letöltése után a feltárás vagy **feltáró adatelemzés** (Exploratory Data Analysis, EDA) következett. A bemenetet a nyers adathalmaz képezte, míg a kimenet az előző fejezetben ismertetett részletes információk és az ezek alapján meghozott tisztítási döntések voltak.

A tisztítás során ezen döntések kivitelezése történt meg, amely javarészt adatok eltávolításával, helyettesítésével és átalakításával járt. Az előfeldolgozási lépések végrehajtását egy ellenőrzési szakasz követte, amely során a tisztítás sikeressége és a kívánt tulajdonságok megőrzése került vizsgálatra.

A sikeres tisztítás után került sor a tokenizáló modell tanítására, majd annak validálására. A validálás célja a megfelelő szókészlet ellenőrzése, valamint a klasszifikációs modell helyes paraméterezésének támogatása volt.

3.3.3. Előfeldolgozás implementálása

Az előfeldolgozás reprodukálhatósága érdekében a felhasznált eszközök kerülnek ismertetésre. Maga a folyamat Jupyter-jegyzetfüzetekben került implementálásra. Ezek megnyitásához szükség volt egy megfelelő környezetre, amely lehetett például a Jupyter Notebook, a Google Colab, vagy a Visual Studio Code megfelelő bővítményekkel. A kód Python programozási nyelven készült, és a fejlesztéshez használt verzió a 3.10.0 volt.

A kód futtatásához a következő, pip-pel telepíthető külső csomagok kerültek felhasználásra (zárójelben a dokumentációban feltüntetett verziószámokkal):

- pandas (2.3.3);
- matplotlib (3.10.8);

- scikit-learn (1.7.2);
- sentencepiece (0.2.1);
- numpy (2.2.6).

A jegyzetfüzet-környezet használatához a **jupyter** csomag telepítése volt szükséges. Emellett legalább az **ipykernel** (7.2.0) csomag is kellett.

3.3.4. Feltáró adatelemzés, adattisztítás és ellenőrzés

A fejezet célja az adathalmaz szerkezetének és tulajdonságainak vizsgálata, valamint az előfeldolgozási döntések bemutatása. A feltáró adatelemzésért felelős notebook a `1-eda-1.ipynb`, míg az adattisztítást végző notebook a `2-data-cleaning.ipynb` fájlban található. Az első notebook bemenetét a nyers adathalmaz képezi, kimenete pedig a rétegezetten felosztott adathalmazok és az előfeldolgozási döntések dokumentálása. Az adattisztító notebook bemenetként a felosztott tanító-, validációs- és tesztkészleteket használja, kimeneteként pedig a tisztított adathalmazok állnak elő.

Első lépésként megtörtént az adathalmaz **rétegezett felosztása** (stratified split) tanító (train), validációs (validation) és teszt (test) részhalmazokra. A rétegezett felosztás biztosítja, hogy a spam és ham címkék aránya minden részhalmazban közel azonos maradjon. Ez különösen fontos bináris osztályozási feladatok esetén, mivel az osztályeloszlás torzulása kedvezőtlenül befolyásolhatja a modell tanítását és kiértékelését.

Az adattisztítás során a dátum oszlop eltávolításra került, mivel nem hordoz releváns információt a spam-osztályozási feladat szempontjából. A tárgymező (subject) az üzenet törzséhez került hozzáfűzésre annak érdekében, hogy a modell az e-mail teljes szöveges tartalmát egységes reprezentációként kezelhesse. A hiányzó értékeket és duplikált rekordokat tartalmazó adatsorok száma elhanyagolható volt, ezért ezek eltávolítása nem eredményezett jelentős adatvesztést. Az osztályeloszlás az előfeldolgozási lépések után is közel változatlan maradt.

Az üzenethosszak karakteralapú eloszlása erősen jobbra ferde volt, ezért a kiugró értékek meghatározása a **Tukey interkvartilis módszer** (Tukey's Interquartile Range Method) alapján történt. A módszer az interkvartilis tartomány segítségével határozza meg az alsó és felső határokat a kiugró értékek azonosítására. A karakterhossz jó közelítést ad az üzenetek információtartalmára, ezért az extrém rövid és extrém hosszú üzenetek eltávolítása indokolt volt.

Az alsó határ esetében a módszer negatív értéket eredményezett, amely gyakorlati szempontból nem értelmezhető, ezért minimális küszöbértékként 15 karakter került meghatározásra. Ennél rövidebb üzenetek biztosan nem tartalmaznak elegendő szemantikai információt az osztályozáshoz. A felső határ újraszámítása a tanítóhalmazon történt, amely alapján a végső küszöbérték 2452 karakter lett. Ennek megfelelően eltávolításra kerültek a 15 karakternél rövidebb és a 2452 karakternél hosszabb üzenetek.

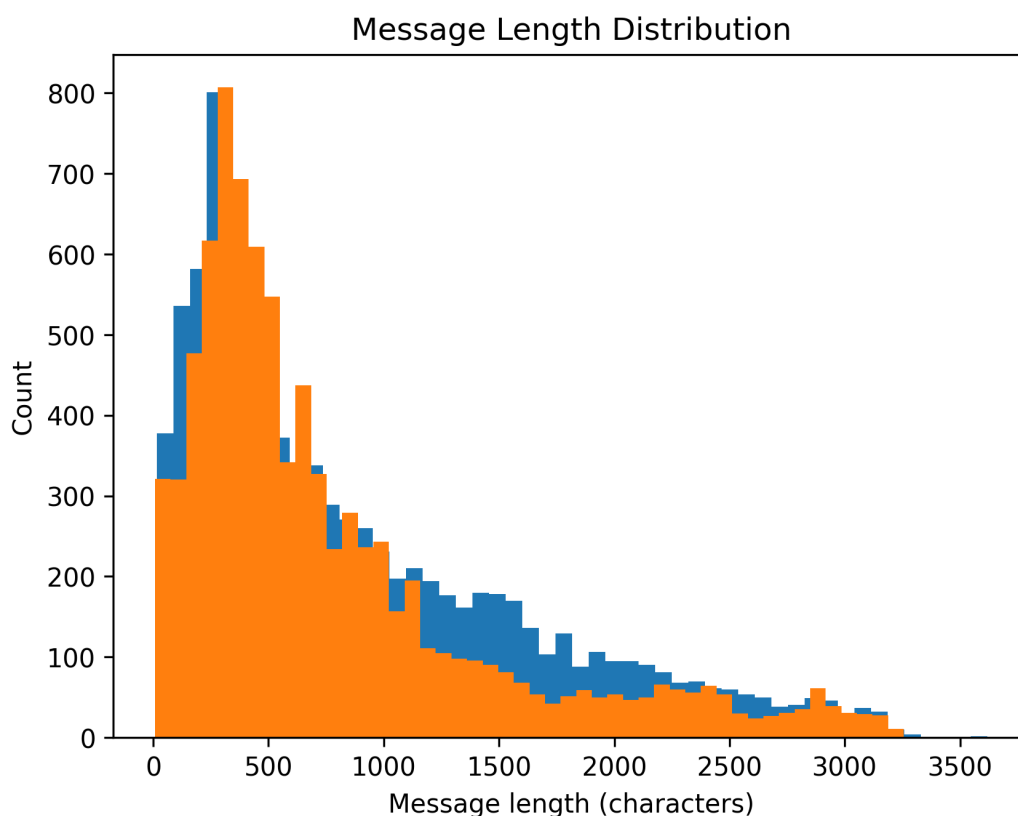
Az URL-ek, e-mail címek, telefonszámok és számok maszkolásra kerültek speciális tokenek segítségével (<URL>, <MAIL>, <PHONE>, <NUM>). A maszkolás célja a szókészlet méretének csökkentése úgy, hogy közben az adott mintázatok szemantikai szerepe megmaradjon.

Az ismétlődő karakterek egységesítése szintén a szókészlet méretének csökkentését szolgálta. Amennyiben egy karakter legalább háromszor ismétlődött egymás után, az

ismétlések száma kettőre került csökkentésre. A speciális karakterek végül nem kerültek eltávolításra, mivel jelenlétük fontos információt hordozhat a kéretlen üzenetek felismerése során.

A mintázatok helyettesítése reguláris kifejezések alkalmazásával történt, amelyek a konfigurációs fájlban kerültek rögzítésre. A helyettesítési folyamat nehézségét az jelentette, hogy az adathalmazban számos speciális karakter körül szóközök jelentek meg, amelyek akadályozták a reguláris kifejezések pontos működését. Ennek kezelésére további normalizációs lépések történtek, amelyek eltávolították a felesleges szóközöket, majd a mondatvégi írásjelek után újra beszúrták azokat.

Az előfeldolgozás eredményeinek ellenőrzése a `3-verification.ipynb` notebookban történt. Az eredeti 33 716 üzenetből összesen 4 311 került eltávolításra, így a végleges korpusz 29 405 üzenetet tartalmazott. A spam és ham címkék aránya mindhárom részhalmazban egy százalékon belüli eltérést mutatott, valamint az üzenethossz-eloszlás alakja sem változott jelentősen. A tanítóhalmaz előfeldolgozás utáni karakterhossz-eloszlását a 3.3. ábra szemlélteti.



3.3. ábra: Előfeldolgozás utáni üzenethossz-eloszlás karakterszám alapján.

Megfigyelhető, hogy az előfeldolgozás után néhány, az eredeti felső határnál hosszabb üzenet továbbra is jelen van. Ennek oka, hogy bizonyos mintázatok maszkolása megnövelte az üzenetek karakterszámát. Például nagyszámú numerikus érték esetén minden szám a `<NUM>` tokenre került cserélésre, amely hosszabb karakterláncot eredményezett.

Az utolsó lépés a szókészlet méretének becslése volt. A becslés az összefüggő alfanumerikus karaktersorozatok vizsgálata alapján történt. Az adathalmaz összesen 82 789 különböző egyedi alfanumerikus szekvenciát tartalmazott, azonban ezek közül 51 717

darab (62.47%) háromnál kevesebbszer fordult elő. Ez alapján a becsült szókészletméret körülbelül 31072 token lett, amely megfelelő kiindulási alapot biztosított a tokenizer konfigurációjához.

3.3.5. Tokenizálás és validálása

A tisztítás után következett a tokenizáció, amelynek forráskódja a 4-tokenization.ipynb fájlban található. A notebook bemenetét a feldolgozott tanító-, validációs- és tesztadathalmazok képezték. Kimenatként előállításra került a SentencePiece tokenizer modellje, a generált tokenkészlet, valamint az egyes szövegek tokenazonosító-sorozatai pickle formátumban.

A tokenizáló tanítását a train metódus biztosította, az alábbi kódrészleten látszódik, hogy ebben a lépésben került meghatározásra a szókészlet mérete, melynek értéke 30.000 (különböző) token volt. A `user_defined_symbols` paraméterben kerültek megadásra a speciális szimbólumok, amelyeket a tokenizáló modellnek nem kellett tovább bontania. Az Unigram modell előnye az volt, hogy az ismeretlen szavakat jól kezelte, rugalmas részsavas szegmentálást biztosított, és az `character_coverage` 1.0 értéke biztosította, hogy az összes karakter feldolgozásra került.

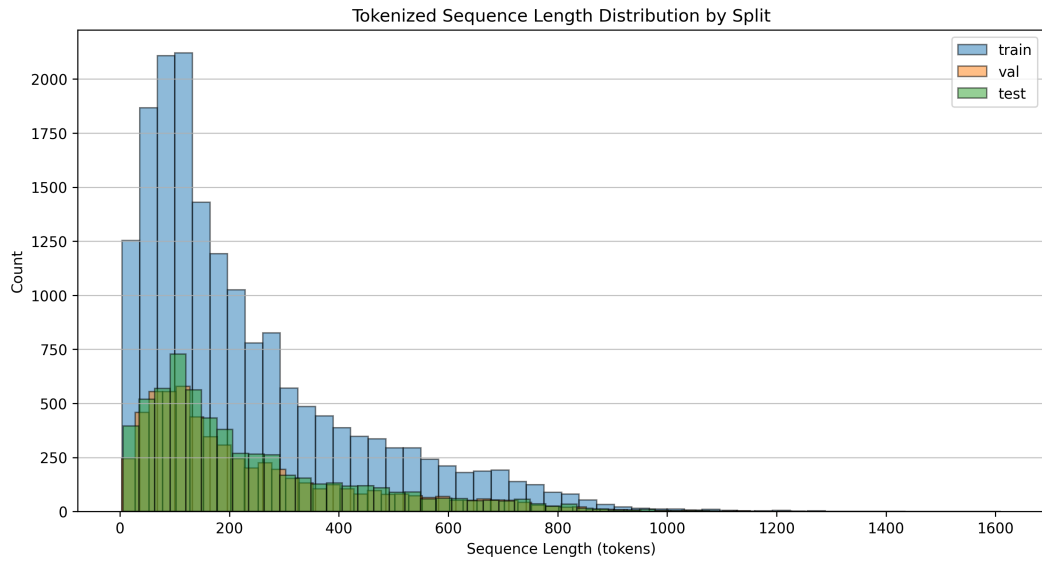
Programkód 3.1. Példa: tokenizálás memóriában

```
spm.SentencePieceTrainer.train(
    input=str(PATHS.lstm_tokenizer_train_data),
    model_prefix=tokenizer_model_prefix,
    vocab_size=30000,
    user_defined_symbols=["<MAIL>", "<URL>", "<NUM>", "<PHONE>"],
    model_type='unigram',
    character_coverage=1.0
)
```

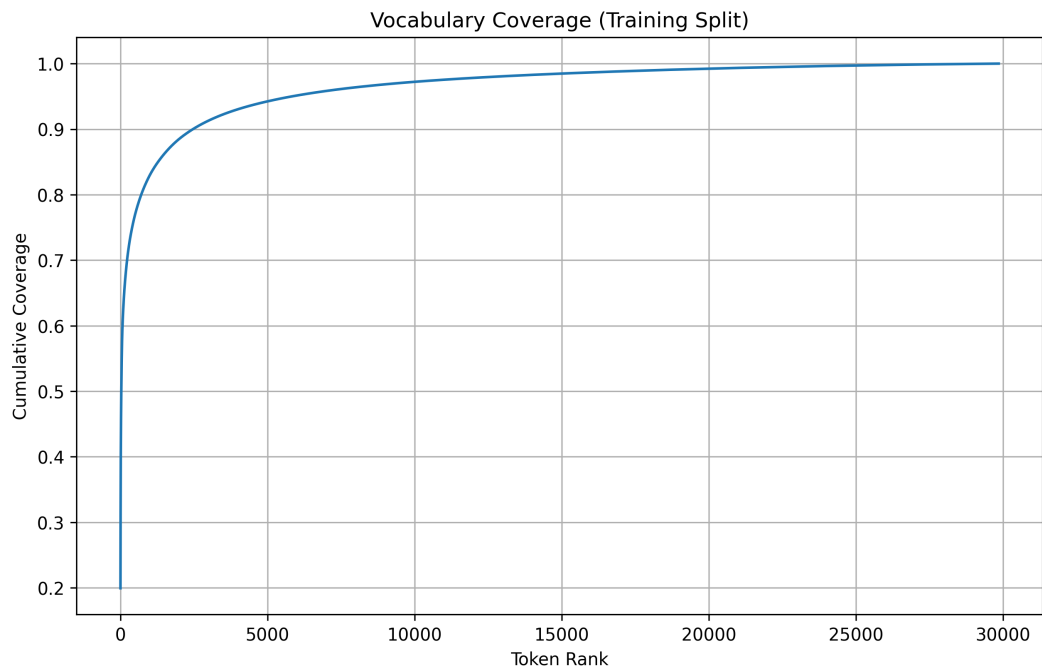
A tokenizált adathalmazok `pickle` formátumban kerültek mentésre, így könnyen visszatölthetők voltak memóriába. Az `ids.pkl` fájlok a numerikus azonosítókat, a `pieces.pkl` fájlok pedig a szöveges tokenrészleteket tárolták. Az objektumok lemezre mentésével nem kellett a tokenizálást minden futtatásnál újra elvégezni.

A tokenizálás validálásához a `./notebooks/5-validation.ipynb` fájlt vizsgáltuk meg, ahol a három adathalmaz hossz eloszlása szintén kirajzolásra került, itt azonban a tokenek száma alapján, és az A 3.4 ábrán látható.

A tokenizált tanítóhalmazon szókészlet-lefedettség elemzés is készült. Az elemzés során meghatározásra kerültek az egyes tokenek előfordulási gyakoriságai, majd kiszámításra került a kumulatív lefedettségük. Az eredmények alapján megfigyelhető volt, hogy a leggyakoribb tokenek viszonylag kis része a teljes tokenállomány jelentős részét lefedte, ami összhangban állt a természetes nyelvekre jellemző Zipf-eloszlással. Ezzel alátámasztva a tokenizálás sikerességét, azonban a 30 leggyakoribb token is kiíratásra került, mert fontos volt ellenőrizni a gyakori tokenek jelentését. Nem meglepő módon a leggyakoribb token a szóköz karakter volt, amely aláhúzással lett reprezentálva. Az első tíz között sok speciális karakter található volt még, mint például a vessző és kettőspont, de itt szerepelt a szám- és URL-maszk is. A továbbiak között már megjelentek a rövidebb gyakori szavak és szórészek, mint a `your`, `of`, `_to`, `_ing`. Az első harminc token a tanító adathalmazban megfelelt a lefedettség ötven százalékának.



3.4. ábra: A tokenizált adatok hosszának eloszlása token szám alapján.



3.5. ábra: A szókincs lefedettség elemzése a tanító halmazon.

A gyakori, jelentéssel nem bíró szavak, az irodalomban **stop-szavak** (Stop Word), nem kerültek eltávolításra. Helyes használatuk információt hordoz az üzenet jó szándékáról, így a gyakori tokenek nagy része ilyen stop-szó.

4. fejezet

A klasszifikációs modell és tanítási folyamat

4.1. Modelltervezés

A dolgozatban alkalmazott klasszifikációs modell egy kétirányú hosszú rövid távú memóriával rendelkező neurális architektúra, amely bináris szövegklasszifikációs feladatot valósít meg. A modell célja annak meghatározása, hogy egy elektronikus levél kéréslen vagy valódi üzenet kategóriába tartozik-e.

A modell bemenetét tokenazonosítók sorozata képezi, amelyeket a korábban ismertetett SentencePiece tokenizáló állít elő. Kimenete pedig egy 0 és 1 közé eső szám, az annak a valószínűsége, hogy a szekvencia kéréslen üzenet. Egy küszöbvel (értéke 0.5) bináris döntést hozunk. Ha a valószínűség nagyobb, mint 0.5, akkor kéréslen az üzenet.

4.1.1. Modell architektúra

A kéréslen üzenetek bináris osztályozására egy kétirányú hosszú rövid távú memória neurális hálón alapuló architektúra használata mellett esett a döntés. A modell célja a szöveges üzenetek szekvenciális reprezentációjának hatékony feldolgozása a szövegkörnyezet mindkét irányból való figyelembevételével.

A bemenetek változó hosszúságúak, ezért feldolgozás során **kitöltés** (padding) alkalmazása szükséges, és ezek a kitöltő tokenek ne járulhassanak hozzá a tanulható tokenekhez. A beágyazási réteg feladatai közé tartozik ezen tokenek kezelése, mely az első réteg az architektúrában. Ebben a rétegben történik a token azonosítók egész számokból való kontextusban gazdag vektorra alakítása. Ezt követően jönnek a hosszú rövid távú memória rétegek, ezek kimenete az utolsó réteg mindkét irányú rejtett állapotainak összefűzése.

Ezt a kimenetet két **teljesen összekapcsolt** (Dense) lineáris réteg követi, amelyek feladata az osztályozás; közöttük egy **egyenirányított lineáris egység** (Rectified Linear Unit, ReLU) aktivációs függvény és egy **lemorzsolás** (**dropout**) regularizáció kerül alkalmazásra a nemlinearitás biztosítása és a túlilleszkedés csökkentése érdekében, a modell általánosító képességének javítására. A második teljesen összekapcsolt réteg kimenete egyetlen neuront tartalmaz, amely logit értéket állít elő.

4.1.2. Hiperparaméter-tér és optimalizálási stratégia

A megfelelő paraméterek kiválasztása az F1 érték maximalizálása szerint történik random hiperparaméter-keresés során az előre definiált értéktartományokból [1] pszeudo-véletlenszerűen kerül kiválasztásra az érték minden paraméter esetében a reprodukálhatóság érdekében. Az optimalizáció alapértelmezett célfüggvénye az F1 mérőszám, melynek jelentősége kiegyensúlyozatlan osztályozás esetén nagyobb. Helyes választás lehet a pontosság vagy a visszahívás is, a megoldandó feladattól függően. Magas pontosság esetén, ha a "pozitív" osztályunk a kéretlen üzenet, kevés normális üzenet kerül a spam mappába, ez a valósághűbb működés. Míg a visszahívás arra ad választ, hogy a valódi kéretlen üzenetek mekkora részét találja meg a modell. Az F1-érték melletti döntést az általános kéretlen üzenetek detektálása támasztja alá.

Ezekkel a paraméterekkel egy új modell kerül betanításra és kiértékelésre. A tanítás folyamata a következő alfejezetben kerül bemutatásra.

A cél a keresési tér kialakításakor olyan paramétertartományok meghatározása, amelyek megfelelő egyensúlyt biztosítanak a modell reprezentációs képessége, a tanítás stabilitása és a számítási költség között. Az alább megadott konkrét numerikus tartományok a jelen dolgozat kísérleti keresési terét jelentik, a hivatkozott irodalban elsősorban az egyes hiperparaméterek hatásmechanizmusáról és gyakorlati hangolási elveiről szerezhet tudást az olvasó.

- **Maximális szekvenciahossz** A modell által feldolgozható tokenek maximális számát határozza meg. Az alsó határnak olyan értéknek kell lennie, amely még elegendő kontextust biztosít a szöveg reprezentálásához, a maximális hossz növeli a memóriaigényt és a számítási költséget. A 3.4 ábra alapján ez az érték **500–1000** token körül helyezkedhet el. Habár 800 token hossz felett már elenyésző az üzenetek száma.
- **Batch méret** Az egy iteráció során feldolgozott minták számát jelenti. A minimális értéket a tanítás stabilitása és a túl zajos gradiensfrissítések elkerülése indokolja, míg a maximális értéket a rendelkezésre álló memória és a generalizációs képesség korlátozza. Ebben a dolgozatban **32, 64, 128** közötti batch méret került vizsgálatra [2].
- **Szóbeágyazási dimenzió** A tokenek vektorreprezentációjának méretét adja meg a **beágyazási dimenzió** (embedding dimension). Méretének elegendőnek kell lennie a megfelelő szemantikai reprezentációhoz, de túl nagy dimenziószám esetén nő a túlilleszkedés veszélye és megnő a paraméterszám. Ez a minimum- és maximumérték-elvárás a következő két rétegre is igaz, így nem lesz ismételve. Vizsgált dimenziószámok: **64, 128, 256** ¹.
- **Rejtett réteg mérete** A hosszú rövid távú memória rejtett rétegeinek dimenzióját határozza meg, azaz konkrétan az LSTM-réteg dimenzióját. Minimális és maximális értéke ugyanazt a kockázatot hordozza magában, mint a szóbeágyazási dimenzió. A vizsgált dimenziószámok is hasonlóak **64, 128, 256, 384** ².
- **Rejtett rétegek száma** Az egymásra épített LSTM-rétegek számát jelenti. Az alsó határ jellemzően **1** réteg, a felső értéket pedig a túlilleszkedés veszélye és a számítási költség korlátozza. Ebben a dolgozatban **1, 2 és 3** réteges konfigurációkat vizsgáltunk ³.

- **Dropout arány** A dropout a regularizáció egyik módszere, amely során a tanítás alatt a neuronok egy része véletlenszerűen deaktiválásra kerül. Alkalmazásának célja a túlilleszkedés csökkentése és a modell általánosító képességének javítása. Túl alacsony érték esetén a regularizáció hatása elhanyagolható lehet, míg túl magas dropout arány mellett a modell tanulási képessége jelentősen romolhat. Ebben a dolgozatban **0.15–0.55** közötti dropout tartományt alkalmaztunk, az alsó felső határ egy kicsit alacsonyabb mint a referencia, de nagyságrendben megegyezik [2].
- **Teljesen összekapcsolt rejtett réteg mérete** Az osztályozó komponens **sűrű rétegének** (dense layer) dimenzióját határozza meg. Feladata az LSTM által előállított reprezentáció további feldolgozása és az osztályozási döntés támogatása. A túl alacsony dimenzió korlátozhatja a modell reprezentációs képességét, míg túl nagy érték esetén növekedhet a túlilleszkedés és a számítási költség. Tipikus választások a **32, 64, 128** dimenziók ⁴.
- **Tanulási ráta** A tanulási ráta határozza meg a paraméterfrissítések lépésközét az optimalizáció során. Túl alacsony érték esetén a modell konvergenciája lassúvá válhat, míg túl magas tanulási ráta instabil tanítást vagy divergenciát eredményezhet. A jelen kísérletekben a tanulási rátát 5×10^{-5} és 3×10^{-3} közötti intervallumban mintavételeztük, logaritmikus skálán [5].
- **Maximális gradiensnorma** A maximális gradiensnorma a gradiensklippelés felső korlátját adja meg, amely elsősorban a robbanó gradiens probléma kezelésére szolgál visszacsatolt neurális hálózatok esetén. Túl alacsony érték mellett a tanulás lelassulhat, míg túl magas korlát esetén a klippelés hatása elenyészővé válhat. Ebben a dolgozatban a gradiensnorma felső korlátját **0.2–3.0** közötti tartományból került kiválasztásra [18].
- **Epochok száma** Az epochok száma azt határozza meg, hogy a modell hányszor dolgozza fel a teljes tanítóhalmazt a tanítás során. Túl alacsony epochszám esetén a modell alultanulhat, míg túl magas érték mellett növekedhet a túlilleszkedés veszélye és a tanítási idő. A jelen kísérletekben **2–5** epoch közötti tartomány került vizsgálatra, korai megállítási és validációs teljesítmény monitorozása mellett ⁵.

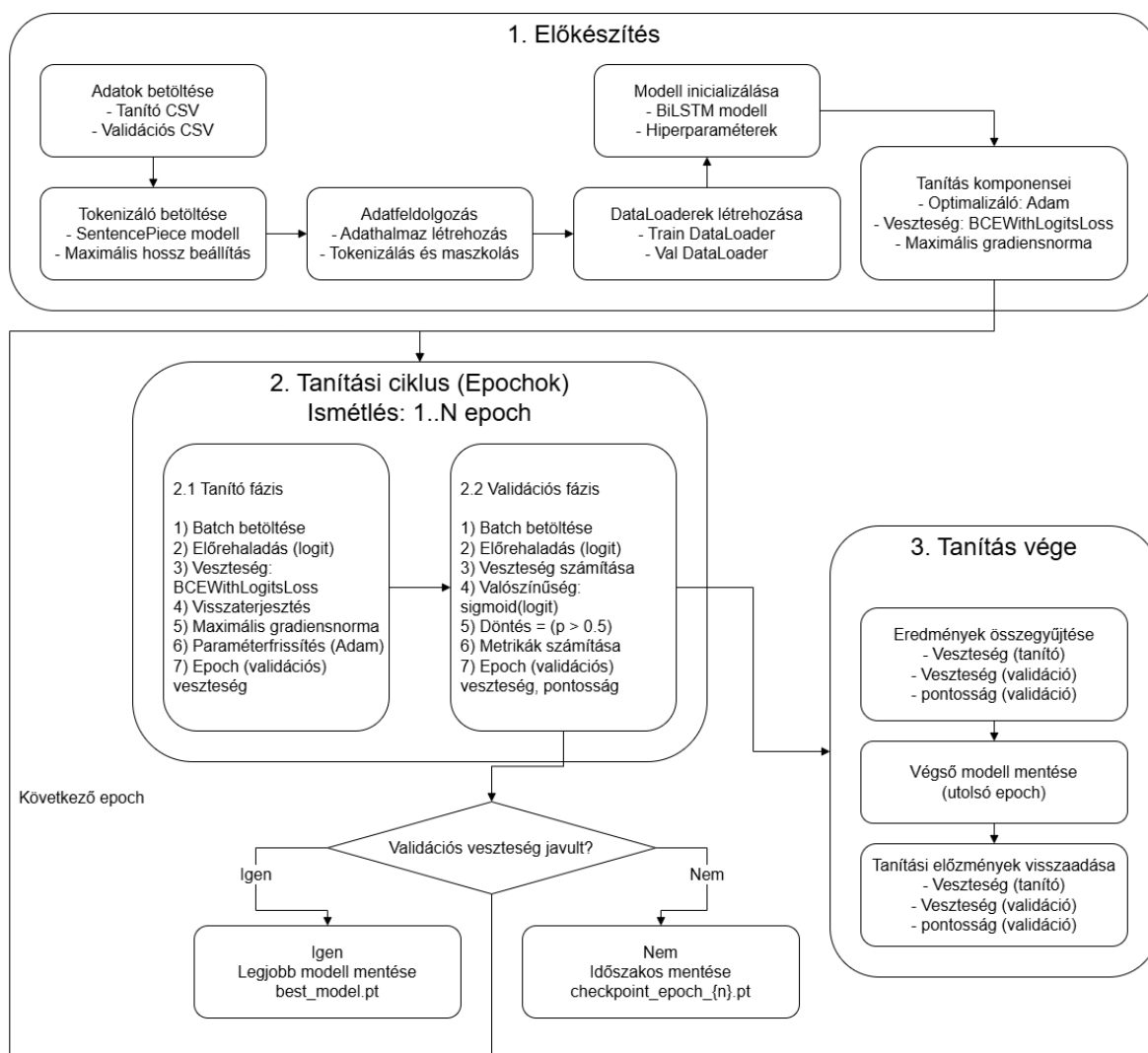
A diszkrét hiperparaméterek esetében előre definiált értékkészletből történt a mintavételezés, míg a folytonos paramétereknél intervallumalapú keresést alkalmaztunk. A tanulási ráta logaritmikus skálán került mintavételezésre, mivel annak optimális értéke jellemzően több nagyságrendet is lefedhet.

Tanítási folyamat, hangolás és tanítóhalmaz növelés

A tanítási folyamat tulajdonképpen koncepcionálisan teljesen megegyezik a hangolás és a növekményes tanítóhalmaz vizsgálata között. Az eltérés logikusan abból áll, hogy

⁴A rétegek méreteit és számosságát, illetve az epochok számát a [12] befolyásolta, mivel az ott megoldott feladat rövid mondatokat osztályozott negatív és pozitív szemantikai jelentés szerint, az üzenetek itt jóval hosszabbak is lehetnek, így a rétegek felső határa magasabb lett a kezdeti keresési térben. Azonban a számítási kapacitás korlátja és a terjedelmes adathalmaz miatt az epochok száma jóval alacsonyabb határon mozog.

a hiperparaméterek keresése során a tanító-, validációs- és tesztadathalmazok változatlanok, míg a hiperparaméterek és az előző fejezetben megállapított rétegek mérete és számossága véletlenszerűen változnak a megadott értéktartományjaiban. Míg a másik esetben a tanítóhalmaz mérete növekszik kumulatíván, más néven halmozódóan, miközben a hiperparaméterek és a többi adathalmaz változatlan marad. A 4.1 ábra szemlélteti a tanítási folyamatot, mely az előkészítés, tanítás és kiértékelés részekből áll.



4.1. ábra: A tanítás folyamata részletesen.

4.2. Implementáció

A modell, a tanítási folyamat és minden egyéb segédprogram implementálása Python nyelven történt, és az egész projekt egységesen a Python 3.10.0 verziót használja. A kódbázis szerkezetét és egyes rövidebb kódrészleteket a BitNinja Technologies Zrt. zárt hozzáférésű projektje inspirálta, melyből részlet nem kerülhet bemutatásra. A kód futtatásához a következő, pip-pel telepíthető külső csomagok kerültek felhasználásra (zárrójelben a dokumentációban feltüntetett verziószámokkal):

- pandas (2.3.3);
- matplotlib (3.10.8);
- scikit-learn (1.7.2);
- sentencepiece (0.2.1);
- numpy (2.2.6).
- torch (2.11.0+cu130);

Továbbá a kód objektumorientált programozási paradigma alapján készült, melynek következtében a kód modularitása kedvezően befolyásolja a kód olvasását, karbantartását és esetleges továbbfejlesztését.

A kezelő felület nagyon egyszerű karakteres felület. Két fő argumentuma a `tune` és a `learning-curve`. A `tune` argumentum esetén a `-num-trials` nevű opcionális argumentum a 20 értéket kapja alapértelmezetten mely a hiperparaméter-keresési próbálkozások száma, míg a `learning-curve` argumentum esetén a `-num-portions` nevű opcionális argumentum a 10 értéket kapja alapértelmezetten, amely a tanítóhalmaz méretének növelése során végzett vizsgálatok száma, és e szám alapján kerül a tanítóhalmaz felosztásra.

4.2.1. Állomány és konfigurációk kezelése

Az állománykezelési műveletek megvalósításáért az **ArtifactManager** osztály felel. Az osztály egységes interfészt biztosít különböző típusú adatok mentésére és betöltésére, beleértve:

- PyTorch modellek és checkpointok mentését,
- JSON konfigurációs állományok kezelését,
- CSV adathalmazok betöltését és mentését,
- grafikonok és ábrák exportálását,
- szöveges állományok kezelését.

Az osztály automatikusan kezeli a könyvtárstruktúra létrehozását, így a mentési műveletek során nem szükséges manuálisan létrehozni a hiányzó mappákat. Az implementáció támogatja a relatív és abszolút útvonalak egységes kezelését is.

A konfigurációk JSON formátumban kerültek tárolásra, melyek közül az egyik feladata az útvonalak és reguláris kifejezések tárolása, míg a másik kettő a hiperparaméterteret és a random hiperparaméter-keresés eredményeit tárolja. Az útvonalak kezelése nem teljes lefedettségű, találni a kódbázisban beégetett vagy kibővített útvonalakat. Az adatok betöltését és mentését külön Python osztályok kezelik.

4.2.2. Modell, hiperparaméterek és tanulási görbe

A modell a PyTorch keretrendszer használatával készült, és a BiLSTMSpamClassifier osztály tartalmazza architektúráját, amely az **előreccsatolás** (forward pass) "forward" nevű metódusából olvasható ki. Ez az egyetlen használt metódus a konstruktoron kívül. Szerkezete a Modell architektúra részben rögzítésre került.

A RandomSearchTuner osztály feladatai közé tartozik a hiperparaméter térből be-töltött különböző érték tartományok kezelése, melyek közé került diszkrét lehetőség, egészszámú vagy lebegőpontos intervallum illetve logaritmikus skálán való mintavételezés. Minden próba esetén egy új pszeudovéletlen hiperparaméter-konfiguráció generálódott, és ezekkel egy új modell tanítása kezdődött.

A tanítás folyamata mind a hiperparaméter mind a tanulási görbe esetében megegyezik, és futtatásukat az LSTMTrainingPipeline osztály indítja. Itt kerül példányosításra az összes különböző osztály, és run() metódusát hívja a mindkét feladat futtatása.

A hiperparaméter-keresés eredményei külön jegyzékekbe kerültek mentésre, és a következő adatokat tartalmazzák:

- a betanított modell állapot mentését,
- a tanítási előzményeket,
- a kiértékelési metrikákat,
- a konfúziós mátrixot,
- valamint a tanulási görbét.

A tanulási görbe osztálya a LearningCurveRunner osztály, feladata a tanítóhalmaz méretének növelése mellett az osztályozási teljesítmény vizsgálata és ezen vizsgálatok mentése vizuálisan és JSON formátumban, a már említett állomány kezelő osztály segítségével.

A kumulatívfelosztást végző külső segédfüggvény érdekessége, hogy a scikit-learn könyvtár StratifiedKFold osztályának split() metódusát alkalmazza, mely pszeudorandom módon megkeveri az adatokat és biztosítja a közel azonos osztályeloszlást a részhalmazok között. Kódja:

5. fejezet

Kísérletek és eredmények

5.1. Előfeldolgozás és tokenizálás

Az előfeldolgozás és tokenizálás részletes szakmai indoklása a korábbi fejezetekben került bemutatásra; ebben a fejezetben ezekre a lépésekre mint kísérleti előfeltételekre hivatkozunk. A kísérletek célja itt már nem a nyelvi normalizálás vagy a tokenizáló algoritmus minősítése, hanem annak vizsgálata, hogy a kiválasztott előfeldolgozási lánc mellett milyen modellkonfiguráció ad stabil, reprodukálható és gyakorlati környezetben is használható osztályozási teljesítményt.

A tokenizált bemenetek hossz-eloszlásának elemzése alapján a szekvenciahossz felső korlátja közvetlenül befolyásolja a memóriaigényt és a futási időt. Ez a kapcsolat a hiperparaméter-optimalizálás során különösen hangsúlyos volt, mert a futásidő-limit nem csupán kényelmi kérdésként jelent meg, hanem ténylegesen meghatározta, hogy egy teljes keresési tér lefuttatható-e a rendelkezésre álló hardveren.

5.2. Hiperparaméter-optimalizálás és tanulási görbe elemzés

Ebben a szakaszban a hiperparaméter-optimalizálás és a kumulatív tanítóhalmazos tanulási görbe elemzésének teljes, többfázisú folyamata kerül bemutatásra. A vizsgálat központi kérdése az volt, hogy hogyan lehet egy erős, de kezdetben instabil és számításigényes konfigurációs teret úgy átalakítani, hogy a tanítás futásideje vállalható maradjon, miközben a modell általánosító képessége ne romoljon. Az optimalizálás célfüggvénye minden fázisban az F1-érték volt, mert spamklasszifikáció esetén a pontosság (precision) és a visszahívás (recall) egyidejű kontrollja szükséges. A kísérleti folyamat három egymást követő fázisból állt:

- **1. fázis:** széles hiperparaméter-tér, megszakított futás (`doc/data/tuning_interrupt`).
- **2. fázis:** szűkített, stabilabb tér (`doc/data/tuning_train_fitted_hyperpar`),
- **3. fázis:** konzervatívabb hiperparaméter-konfiguráció és tanulási görbe (`data/models/tuning`, `data/models/learning_curve`).

5.2.1. Kísérleti környezet, metrika és döntési szabály

A futtatások egyetlen lokális gépen történtek, ezért a számítási költség közvetlenül korlátozta a kipróbálható konfigurációk számát. A mérések során használt hardver és környezet:

- **GPU:** NVIDIA GeForce RTX 5050 Laptop GPU,
- **GPU memória:** 8151 MiB VRAM,
- **Rendszermemória:** 33617924096 byte (kb. 31.3 GiB),
- **Python:** 3.10.0,
- **PyTorch:** 2.11.0+cu130,
- **CUDA runtime:** 13.0.

A kiértékelés minden trial végén azonos döntési küszöbvel (`threshold = 0.5`) történt. Ez az egységes küszöb összehasonlíthatóvá tette a futásokat, ugyanakkor a későbbi elemzésben láthatóvá vált, hogy szélsőséges konfigurációk esetén a fix küszöb nagy kiengéseket is felerősíthet (például túl sok hamis negatív vagy hamis pozitív döntés esetén).

5.2.2. A három fázis keresési tere

A három kísérleti fázis közötti legfontosabb különbség a hiperparaméter-tér fokozatos szűkítése volt. Az 5.1. táblázat összefoglalja a változtatások irányát.

5.1. táblázat: A hiperparaméter-tér módosítása a három fázisban.

Hiperparaméter	1. fázis	2. fázis	3. fázis
Max. szekvenciahossz	[512, 800, 1024]	[500, 600, 700, 800]	[500, 600, 700]
Batch méret	[32, 64, 128]	[32, 64, 128]	[32, 64, 128]
Embedding dimenzió	[64, 128, 256]	[64, 128, 256]	[64, 128, 256]
Hidden size	[64, 128, 256, 384]	[64, 128, 256]	[64, 128]
LSTM rétegek száma	[1, 2, 3]	[2, 3]	[2]
Dropout	[0.15, 0.55]	[0.15, 0.50]	[0.15, 0.50]
Dense hidden	[32, 64, 128]	[32, 64, 128]	[32, 64, 128]
Learning rate	[0.00005, 0.003]	[0.00005, 0.003]	[0.00005, 0.002]
Max grad norm	[0.2, 3.0]	[0.5, 3.0]	[0.5, 3.0]
Epochok száma	[2, 3, 4, 5]	[2, 3, 4]	[2, 3]

A táblázatból látható, hogy a második fázisban a szélsőségesen drága és instabil konfigurációk kizárása volt a fő cél, a harmadik fázisban pedig már tudatosan konzeratív, alacsonyabb varianciát adó keresési tér kialakítása történt. Ez a gyakorlatban azt jelentette, hogy a modelleslád expresszivitása nem szűnt meg, de az erősen túlparaméterezett és kis mintán instabil viselkedésre hajlamos kombinációk aránya jelentősen csökkent.

5.2.3. Első fázis: széles keresési tér és megszakított futás

Az első fázis (`doc/data/tuning_interrupt`) célja eredetileg az volt, hogy széles keresési térben felmérhető legyen, milyen teljesítményszint érhető el a rendelkezésre álló architektúrával. A futtatás azonban gyakorlati okból megszakadt: a 10. trial több mint négy órán keresztül futott, ezért a kísérlet manuálisan leállításra került. Ez a pont már önmagában fontos eredmény, mert rámutat arra, hogy a keresési tér nemcsak statisztikai, hanem *üzemeltetési* szempontból is optimalizálandó.

A mappában 9 teljesen kiértékelt trial található. Ezek alapján:

- **legjobb F1:** 0.9807 (`trial_006`),
- **legrosszabb F1:** 0.0033 (`trial_007`),
- **átlagos F1:** 0.8514.

Az első fázis egyik legfontosabb tanulsága a szélsőséges variancia. A legjobb és legrosszabb trial közötti különbség több nagyságrendnyi minőségeltérést jelentett. A `trial_007` esete különösen informatív, mert bár a precision érték magas volt, a recall gyakorlatilag összeomlott, ezért az F1 szinte nullához közeli lett. Ez tipikusan azt jelzi, hogy a modell döntési felülete nagyon egyoldalúvá vált, vagyis a konfiguráció a pozitív osztály nagyon kis részét találta meg.

Módszertani szempontból ez a fázis két okból is hasznos maradt a megszakítás ellenére:

1. bizonyította, hogy a keresési tér tartalmaz kiváló konfigurációkat;
2. kimutatta, hogy ugyanebben a térben számos olyan kombináció is van, amely instabil, és a rendelkezésre álló hardveren aránytalanul drága.

Ennek alapján a második fázis tervezése már célzottan a kockázatos tartományok visszaszorítására épült.

5.2.4. Második fázis: szűkített keresési tér és stabilabb tuning

A második fázisban (`doc/data/tuning_train_fitted_hyperpar`) a keresési tér szűkítésével 20 trial teljes lefutása megtörtént. Ez önmagában is minőségi előrelépés az első fázishoz képest, mert megszűnt az a helyzet, hogy egyetlen trial aránytalanul hosszú futási ideje miatt a teljes keresési ciklus félbeszakadjon.

A 20 trial összesített képe:

- **átlagos F1:** 0.9536,
- **legjobb F1:** 0.9814 (`trial_003`),
- **legrosszabb F1:** 0.8900 (`trial_002`).

A fázis legjobb három konfigurációja:

- `trial_003`: `max_length=800, batch=32, emb=128, hidden=256, layers=3, dropout=0.4333, lr=0.001117, grad_norm=0.899, epoch=3`;
- `trial_014`: `max_length=500, batch=64, emb=128, hidden=64, layers=3, dropout=0.1511, lr=0.001805, grad_norm=1.158, epoch=4`;

- **trial_018:** max_length=600, batch=32, emb=256, hidden=128, layers=2, dropout=0.2428, lr=0.000283, grad_norm=1.029, epoch=4.

Ezek a top konfigurációk fontos közös mintázatot mutatnak:

- egyik sem épít a korábbi legszélsőségesebb méretre (például 384-es hidden),
- a learning rate mindhárom esetben kontrollált tartományban marad,
- a rétegszám 2–3 között stabilan működik.

Ez a fázis tehát azt igazolta, hogy a keresési tér átgondolt szűkítésével egyszerre javítható a gyakorlati futtathatóság és a teljesítmény reprodukálhatósága. Különösen fontos eredmény, hogy a legrosszabb trial is elfogadható F1-et adott, vagyis a futások közötti variancia érdemben csökkent.

5.2.5. Harmadik fázis: konzervatívabb tér és tanulási görbe

A harmadik fázis célja már nem elsősorban a maximális csúcsteljesítmény keresése volt, hanem a kumulatív tanítóhalmazos tanulási görbe stabilizálása. Ehhez a keresési tér tovább szűkült (`data/models/tuning`), majd a kiválasztott konfigurációval a learning curve futtatás a `data/models/learning_curve` mappába készült.

A tuningfázis összesítése ebben a körben:

- **trialek száma:** 20,
- **átlagos F1:** 0.9473,
- **legjobb F1:** 0.9817 (`trial_006`),
- **legrosszabb F1:** 0.8364 (`trial_001`).

A legjobb trial hiperparaméterei:

- max_length=700,
- batch_size=32,
- embedding_dim=64,
- hidden_size=64,
- num_layers=2,
- dropout_rate=0.4948,
- dense_hidden=32,
- learning_rate=0.001222,
- max_grad_norm=1.450,
- num_epochs=3.

Ez a konfiguráció jól illeszkedik a konzervatívabb keresési stratégiához: kisebb hidden méret, fixen két LSTM réteg, rövidebb tanítási ciklus.

Tanulási görbe: részhalmazonkénti eredmények

A kumulatív tanítóhalmaz-növelés 10 részhalmazos futásban készült. Az 5.2. táblázat a fő metrikákat tartalmazza.

5.2. táblázat: Tanulási görbe metrikák részhalmazonként (kumulatív tanítóhalmaz).

Részhalmaz	Corpus size	Precision	Recall	F1
portion_01_of_10	1739	0.8814	0.7903	0.8334
portion_02_of_10	3478	0.8988	0.9356	0.9168
portion_03_of_10	5217	0.9307	0.9620	0.9461
portion_04_of_10	6956	0.9658	0.9049	0.9344
portion_05_of_10	8695	0.9585	0.9696	0.9640
portion_06_of_10	10434	0.9595	0.9538	0.9566
portion_07_of_10	12173	0.9801	0.9581	0.9689
portion_08_of_10	13911	0.9736	0.9762	0.9749
portion_09_of_10	15649	0.9705	0.9663	0.9684
portion_10_of_10	17387	0.9665	0.9812	0.9738

A táblázat két fontos mintázatot mutat:

1. **globális javulási trend:** a korai részhalmazhoz képest az F1 a teljes tanítóhalmaz közelében érdemben magasabb;
2. **lokális fluktuációk:** a görbe nem szigorúan monoton, de a második felében tartósan magas teljesítményszintet tart.

Ez gyakorlati szempontból azt jelenti, hogy a modell általánosító képessége növekvő adatmennyiséggel javul, miközben a kisebb visszaesések már nem jelentenek teljesítmény-összeomlást, csak lokális varianciát. Az utolsó három részhalmaz F1-értékei közel vannak egymáshoz, ezért a görbe a végső szakaszban platójellegű viselkedést mutat.

5.2.6. A 0.5-ös küszöb és az instabil konfigurációk hatása

A fix `threshold = 0.5` beállítás minden futásban egységes értékelést biztosított. Ugyanakkor az első fázisban látott szélsőséges esetek rámutattak, hogy instabil konfigurációknál a küszöb hatása felerősödhet: a modelpredikciók eltolódása miatt nagyon aszimmetrikus precision–recall párok is megjelenhetnek. Ez különösen akkor kritikus, ha a modell egy osztály felé torzul.

A második és harmadik fázis eredményei alapján viszont látható, hogy a probléma elsődlegesen nem a küszöbben, hanem a túl széles és részben agresszív hiperparaméter-térben volt. Amint a keresési tér konzervatívabbá vált, a 0.5-ös küszöb mellett is lényegesen stabilabb metrikák adódtak. Ezért ebben a dolgozatban a küszöböt nem külön optimalizáltuk, hanem kontrollált konstansként kezeltük.

5.2.7. Összegzés: kompromisszumok és tanulságok

A teljes kísérletsor alapján a következő következtetések fogalmazhatók meg:

- **Futásidő kontra keresési szélesség:** a túl széles tér gyakorlati futtatásban nem hatékony, mert hosszú és instabil trialeket generálhat.
- **Stabilitás kontra csúcsteljesítmény:** a konzervatívabb tér kis mértékben szűkíti a potenciális maximumot, de jelentősen javítja a reprodukálhatóságot.
- **Generalizáció kontra komplexitás:** a kisebb és visszafogottabb konfigurációk a tanulási görbében kiegyensúlyozottabb trendet adtak.
- **Metrika-választás:** az F1 célfüggvényként jól működött, mert egyszerre érzékeny a precision és recall változására.

Összességében a többfázisú optimalizálási stratégia nemcsak a legjobb trial megtalálását szolgálta, hanem egy olyan kísérleti protokoll kialakítását is, amely hardverkorlát mellett is megbízható. A fő eredmény ezért nem pusztán egyetlen maximális F1-érték, hanem az a folyamat, amelyben a keresési tér, a futási költség és a teljesítménystabilitás közötti kompromisszum kontrollálhatóvá vált.

5.2.8. Korlátok és továbbfejlesztési lehetőségek

A bemutatott eredmények mellett fontos a korlátok rögzítése:

- a futások egyetlen hardverkörnyezetben történtek;
- a trialek száma véges, ezért ritka, de jó konfigurációk kimaradhatnak;
- a learning curve egyetlen futásra épül részhalmazonként, ismétlések nélküli átlagolással;
- a döntési küszöb fixen 0.5 maradt.

A következő fejlesztési lépések szakmailag indokoltak:

1. több seedes futtatás és átlagolt learning curve;
2. validációs küszöboptimalizálás (precision–recall kompromisszum mentén);
3. erőforrás-függő, adaptív keresési stratégia (korai leállítással);
4. a legjobb konfigurációk környezetében finomabb, lokális hiperparaméter-keresés.

Ezekkel a lépésekkel a dolgozatban felépített keretrendszer könnyen továbbvihető lenne ipari vagy kutatási környezetben is, anélkül hogy az eddigi eredmények reprodukálhatósága sérülne.

6. fejezet

Összefoglalás és továbbhaladási irány

Források

- [1] James Bergstra és Yoshua Bengio. „Random search for hyper-parameter optimization”. *J. Mach. Learn. Res.* 13.null (2012. febr.), 281–305. ISSN: 1532-4435.
- [2] Liriam Enamoto és tsai. „Multi-label legal text classification with BiLSTM and attention”. *International Journal of Computer Applications in Technology* 68 (2022. jan.), 369. old. DOI: 10.1504/IJCAT.2022.125186.
- [3] IBM. *Artificial intelligence*. URL: <https://www.ibm.com/think/topics/artificial-intelligence> (elérés dátuma 2026. 04. 29.).
- [4] IBM. *What is a confusion matrix?* International Business Machines Corporation (IBM). URL: <https://www.ibm.com/think/topics/confusion-matrix> (elérés dátuma 2026. 05. 11.).
- [5] IBM. *What is learning rate in machine learning?* International Business Machines Corporation (IBM). URL: <https://www.ibm.com/think/topics/learning-rate> (elérés dátuma 2026. 05. 12.).
- [6] Kamran Kowsari és tsai. „Text classification algorithms: A survey”. *Information* 10.4 (2019), 150. old. DOI: 10.3390/info10040150. URL: <https://doi.org/10.3390/info10040150>.
- [7] Taku Kudo és John Richardson. „SentencePiece: A simple and language independent subword tokenizer and detokenizer for Neural Text Processing”. *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. Szerk. Eduardo Blanco és Wei Lu. Brussels, Belgium: Association for Computational Linguistics, 2018. nov., 66–71. old. DOI: 10.18653/v1/D18-2012.
- [8] Qian Li és tsai. „A Survey on Text Classification: From Traditional to Deep Learning”. *ACM Transactions on Intelligent Systems and Technology* 13.2 (2022), 1–39. old. DOI: 10.1145/3495162.
- [9] Gang Liu és Jiabao Guo. „Bidirectional LSTM with attention mechanism and convolutional layer for text classification”. *Neurocomputing* 337 (2019), 325–338. old. ISSN: 0925-2312. DOI: <https://doi.org/10.1016/j.neucom.2019.01.078>. URL: <https://www.sciencedirect.com/science/article/pii/S0925231219301067>.
- [10] Vangelis Metsis, Ion Androutsopoulos és Georgios Paliouras. „Spam Filtering with Naive Bayes - Which Naive Bayes?": 2006. jan.
- [11] Procogia. *The Interquartile Range Method For Outliers*. URL: <https://procogia.com/interquartile-range-method-for-reliable-data-analysis/> (elérés dátuma 2026. 05. 04.).

- [12] Hao Qin. „BiLSTM Text Classification Incorporating Attentional Mechanisms”. *Academic Journal of Computing & Information Science* 6.13 (2023), 92–98. old. DOI: 10.25236/AJCIS.2023.061314. URL: <https://francis-press.com/uploads/papers/louD4xcIlZAI6NdXM0126npbxAqYc0cxIOMK41gW.pdf>.
- [13] Cole Stryker és Jim Holdsworth. *What is NLP?* International Business Machines Corporation (IBM). URL: <https://www.ibm.com/think/topics/natural-language-processing> (elérés dátuma 2026. 04. 29.).
- [14] Kamal Taha és tsai. „A comprehensive survey of text classification techniques and their research applications: Observational and experimental insights”. *Computer Science Review* 54 (2024), 100664. old. DOI: 10.1016/j.cosrev.2024.100664. URL: <https://doi.org/10.1016/j.cosrev.2024.100664>.
- [15] U.S. Census Bureau. *Data science*. URL: <https://www.census.gov/topics/research/data-science.html> (elérés dátuma 2026. 04. 29.).
- [16] Mengnan Wang. „Research on Text Classification Method Based on NLP”. *Advances in Computer, Signals and Systems* 7 (2023), 93–100. old. DOI: 10.23977/acss.2023.070213.
- [17] M. Wiechmann és M. Noppel. *Enron Spam Dataset*. Data set. GitHub. URL: https://github.com/MWiechmann/enron_spam_data (elérés dátuma 2026. 05. 06.).
- [18] xbeat. *Gradient Clipping and Learning Rate Scheduling in Python*. URL: <https://github.com/xbeat/Machine-Learning/blob/main/Gradient%20Clipping%20and%20Learning%20Rate%20Scheduling%20in%20Python.md> (elérés dátuma 2026. 05. 12.).
- [19] Peng Zhou és tsai. *Text Classification Improved by Integrating Bidirectional LSTM with Two-dimensional Max Pooling*. 2016. arXiv: 1611.06639 [cs.CL]. URL: <https://arxiv.org/abs/1611.06639>.

CD Használati útmutató

Ennek a címe lehet például *A mellékelt CD tartalma* vagy *Adathordozó használati útmutató* is.

Ez jellemzően csak egy fél-egy oldalas leírás. Arra szolgál, hogy ha valaki kézhez kapja a szakdolgozathoz tartozó CD-t, akkor tudja, hogy mi hol van rajta. Jellemzően elég csak felsorolni, hogy milyen jegyzékek vannak, és azokban mi található. Az elkészített programok telepítéséhez, futtatásához tartozó instrukciók kerülhetnek ide.

A CD lemezre mindenképpen rá kell tenni

- a dolgozatot egy `dolgozat.pdf` fájl formájában,
- a LaTeX forráskódját a dolgozatnak,
- az elkészített programot, fontosabb futási eredményeket (például ha kép a kimenet),
- egy útmutatót a CD használatához (ami lehet ez a fejezet külön PDF-be vagy Markdown fájlként kimentve).